

SecureGen: Generador Inteligente de Contraseñas Seguras

Como Respuesta al Impacto de las Nuevas Tecnologías en la Sociedad

Camila Cumbicus Betsabe Cuesta

Facultad de Ingeniería, Universidad Internacional del Ecuador (UIDE)

Materia: Lógica de programación

Estefania Vanessa Heredia Jimenez

27 de junio de 2026

Introducción

Las nuevas tecnologías han transformado de manera profunda la forma en que las personas se comunican, trabajan, estudian y resuelven problemas cotidianos. Internet, los dispositivos móviles y los servicios digitales se han vuelto indispensables, pero ese mismo nivel de dependencia ha incrementado la exposición de las personas a riesgos como el robo de identidad, el acceso no autorizado a cuentas personales y la pérdida de información sensible.

El presente Proyecto Integrador responde a esta problemática mediante el desarrollo de SecureGen, un sistema de consola elaborado en Python que permite generar contraseñas seguras y personalizadas, siguiendo los lineamientos institucionales establecidos para este tipo de proyectos (UIDE, 2026). A diferencia de un generador de contraseñas genérico, SecureGen delimita deliberadamente su alcance a un problema concreto y verificable —la debilidad de las contraseñas elegidas por los propios usuarios— y lo resuelve con una herramienta ejecutable, pequeña y trazable línea por línea, en lugar de abordar la ciberseguridad solo desde la teoría. El proyecto integra los conocimientos adquiridos a lo largo de las cuatro unidades de la asignatura, con énfasis particular en la Unidad 4: Estructuras de Datos y Funciones, cuyos conceptos —tuplas, listas, diccionarios y funciones con parámetros y retorno— constituyen el eje central de la implementación (Luján Castillo, 2019).

En las siguientes secciones se describe el problema abordado y su alcance específico, el propósito del proyecto, su relación con los contenidos del curso, el análisis y diseño del sistema, su implementación, los resultados obtenidos durante las pruebas y la extrapolación lógica que de ellos se deriva, las implicaciones y limitaciones identificadas en detalle, y una reflexión final sobre el impacto de la tecnología en la sociedad.

Descripción del Problema

Uno de los efectos menos visibles —pero más relevantes— del avance tecnológico es el aumento sostenido de los incidentes de ciberseguridad. Gran parte de estos incidentes no se origina en fallas sofisticadas de los sistemas, sino en una causa mucho más simple y extendida: el uso de contraseñas débiles, repetidas entre distintos servicios o fáciles de adivinar (fechas de nacimiento, secuencias numéricas, nombres propios, entre otras).

Esta situación expone a usuarios de todas las edades —desde estudiantes hasta profesionales— a la pérdida de cuentas de correo, redes sociales, banca en línea y plataformas educativas. El problema se

agrava porque muchas personas no cuentan con una herramienta sencilla y accesible que les ayude a generar, en segundos, una contraseña robusta y adaptada a sus propias necesidades (longitud, uso de símbolos, números, etc.). Organismos de referencia en seguridad digital, como el National Institute of Standards and Technology, han actualizado recientemente sus lineamientos para enfatizar que la longitud de una contraseña influye más en su seguridad que el cumplimiento de reglas de complejidad forzadas, y recomiendan priorizar contraseñas largas y aleatorias por sobre combinaciones cortas con caracteres especiales obligatorios (National Institute of Standards and Technology [NIST], 2025).

A partir de este diagnóstico, el proyecto delimita su alcance de forma explícita y acotada, evitando la tentación de abarcar un problema demasiado amplio: desarrollar una aplicación de consola, funcional y de fácil uso, que permita (1) generar contraseñas seguras de forma personalizada según longitud y tipos de caracteres, (2) llevar un registro temporal (historial) de las contraseñas generadas durante una sesión, y (3) validar de forma robusta las entradas del usuario, sin requerir conexión a internet ni dependencias externas más allá de la biblioteca estándar de Python (`random` y `string`).

Este enfoque resulta a la vez creativo y factible dentro del contexto de un curso introductorio de programación: en lugar de abordar la ciberseguridad únicamente desde una perspectiva teórica —un ángulo ya ampliamente documentado—, el proyecto propone una herramienta ejecutable, deliberadamente acotada en alcance pero significativa en su aplicación práctica, que un estudiante de los primeros niveles puede comprender, justificar y modificar línea por línea. Este tipo de soluciones pequeñas pero funcionales constituye un aspecto todavía poco explorado dentro de los proyectos introductorios de programación, en los que suele predominar el ejercicio académico desconectado de una problemática real.

Propósito y Objetivos

Propósito general: contribuir, desde una solución informática sencilla y bien delimitada, a la reducción de un riesgo tecnológico concreto —el uso de contraseñas inseguras— al mismo tiempo que se integran los contenidos de programación trabajados durante el curso.

- Analizar el impacto de las nuevas tecnologías en la sociedad, particularmente en el ámbito de la seguridad digital.
- Aplicar conocimientos de diseño, programación y arquitectura de software en un caso real y acotado.

- Desarrollar un sistema funcional basado en una problemática claramente definida y verificable mediante pruebas.
- Integrar los contenidos de las cuatro unidades de la asignatura, con énfasis en estructuras de datos y funciones.
- Reflexionar sobre el uso y la proyección futura de la tecnología en el entorno real de los usuarios.

Relación con los Contenidos de la Asignatura

Las estructuras de datos en Python —tuplas, listas y diccionarios— permiten agrupar múltiples valores dentro de una sola unidad de almacenamiento, en lugar de representarlos mediante variables aisladas (Luján Castillo, 2019). El desarrollo de SecureGen permitió aplicar de manera articulada los contenidos revisados en las cuatro unidades del curso, tal como se resume en la Tabla 1.

Tabla 1. Relación entre las unidades de la asignatura y su aplicación en SecureGen

Unidad	Contenido aplicado en el proyecto
1	Lógica de programación y sintaxis básica de Python (variables, tipos de datos, entrada/salida con <code>input()</code> y <code>print()</code>).
2	Estructuras de control condicionales: <code>if / elif / else</code> , utilizadas para validar la opción del menú y las preferencias del usuario.
3	Estructuras de control repetitivas (<code>while</code> , <code>for</code>) y manejo de errores con <code>try/except</code> , usadas para validar la longitud de la contraseña, la entrada numérica y recorrer el historial.
4	Estructuras de datos (tupla, lista, diccionario) y funciones con parámetros y valores de retorno: el núcleo del sistema desarrollado.

La integración de estos contenidos no fue acumulativa sino funcional: cada estructura aprendida cumple un rol específico dentro del sistema. Por ejemplo, sin las estructuras condicionales no sería posible validar las preferencias del usuario; sin las estructuras repetitivas no se podría completar la longitud deseada de la contraseña ni reintentar ante una entrada inválida; y sin el diccionario, la lista y la tupla de la Unidad 4, no existiría una forma ordenada de organizar los conjuntos de caracteres, el historial de resultados y los valores de referencia del sistema (Luján Castillo, 2019).

Análisis y Diseño del Software

Antes de implementar el código, se analizaron los actores y las acciones que el sistema debía soportar, y se diseñaron los diagramas de funcionalidad (casos de uso y flujo) y el diagrama de arquitectura que organizó las decisiones de diseño en tres capas: interfaz, lógica y datos, junto con una capa transversal de validación y pruebas.

Diagrama de Casos de Uso

La Figura 1 identifica al actor «Usuario» y las tres acciones principales que ofrece el menú del sistema — generar una contraseña, consultar el historial y salir del programa—. A diferencia de una versión simplificada, este diagrama hace explícitas las relaciones de inclusión (<<include>>): generar una contraseña incluye necesariamente validar la longitud mínima, elegir los tipos de caracteres y registrar el resultado en el historial. También se muestra una relación de extensión (<<extend>>): el caso de uso «Ver historial» se extiende con el comportamiento «mostrar historial vacío» cuando todavía no se ha generado ninguna contraseña en la sesión, lo cual evidencia una dependencia lógica entre ambos casos de uso.

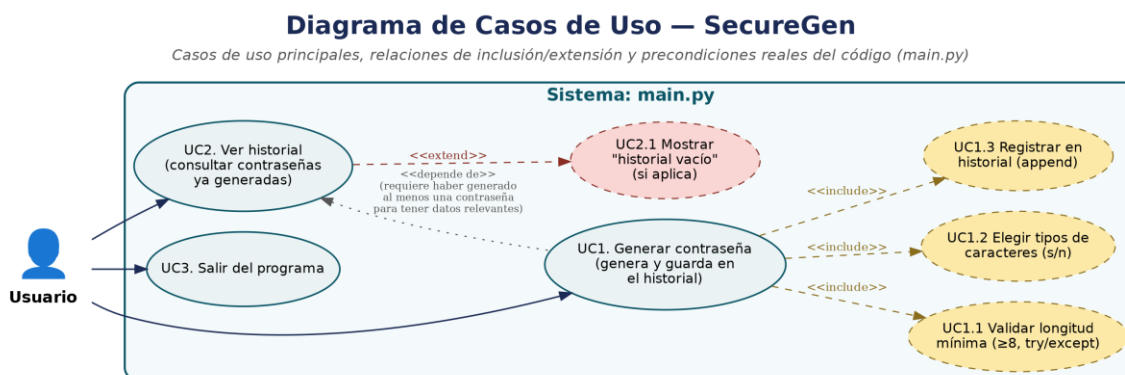


Figura 1. Diagrama de casos de uso del sistema SecureGen, con relaciones de inclusión y extensión.

Diagrama de Flujo

La Figura 2 describe, paso a paso, la lógica real de la función `menu()`: la estructura condicional encadenada `if / elif / elif / else` que evalúa la opción ingresada por el usuario, e incorpora además el detalle interno de las dos validaciones críticas del sistema —el manejo de excepciones `try/except` ante una entrada no numérica y el bucle de reintento cuando la longitud solicitada es menor a 8— así como la lógica interna de `generar_password()` (construcción de grupos de caracteres activos, garantía de al menos un carácter de cada tipo seleccionado, y aleatorización con `random.shuffle()`). El conector "A" indica los puntos en los que el programa retorna al inicio del bucle `while True` para volver a mostrar el menú.

Diagrama de Flujo Detallado — SecureGen (main.py)

Incluye validaciones, manejo de excepciones y la lógica interna de `generar_password()` y `mostrar_historial()`

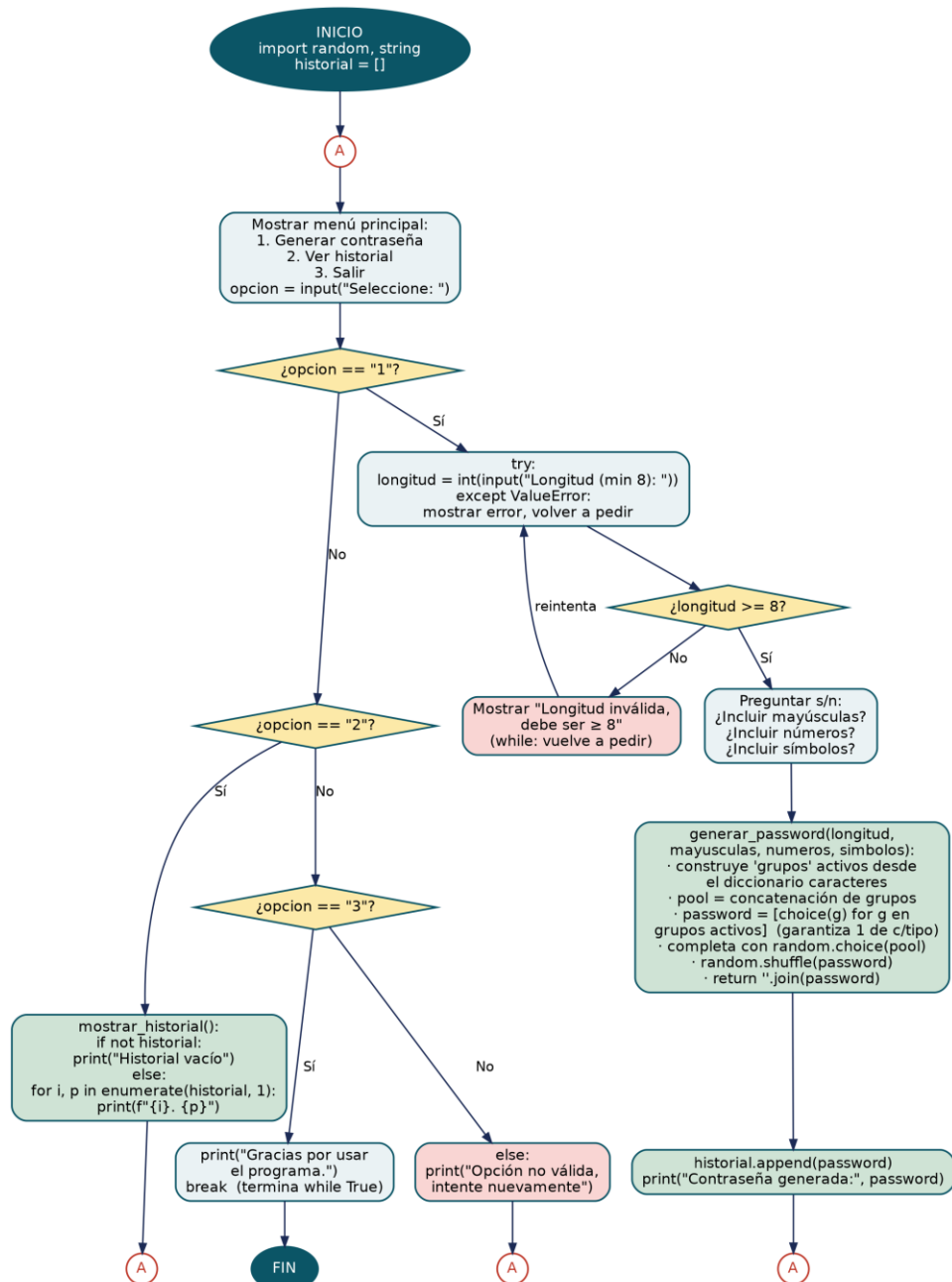


Figura 2. Diagrama de flujo detallado del programa principal (menú), incluyendo validaciones internas.

Diagrama de Arquitectura y Estructuras de Datos

El sistema se organizó conceptualmente en tres capas, lo que facilita su lectura, mantenimiento y futura escalabilidad: una capa de interfaz (función menu, que controla el flujo del programa), una capa de lógica (las funciones generar_password, mostrar_historial y la validación de entrada) y una capa de datos, integrada por las tres estructuras de datos de la Unidad 4. La Figura 3 amplía el diagrama original incorporando la dependencia explícita con la biblioteca estándar de Python (random y string) y una capa transversal de pruebas que valida el comportamiento de menu() y generar_password() frente a los casos descritos en la sección de Resultados.

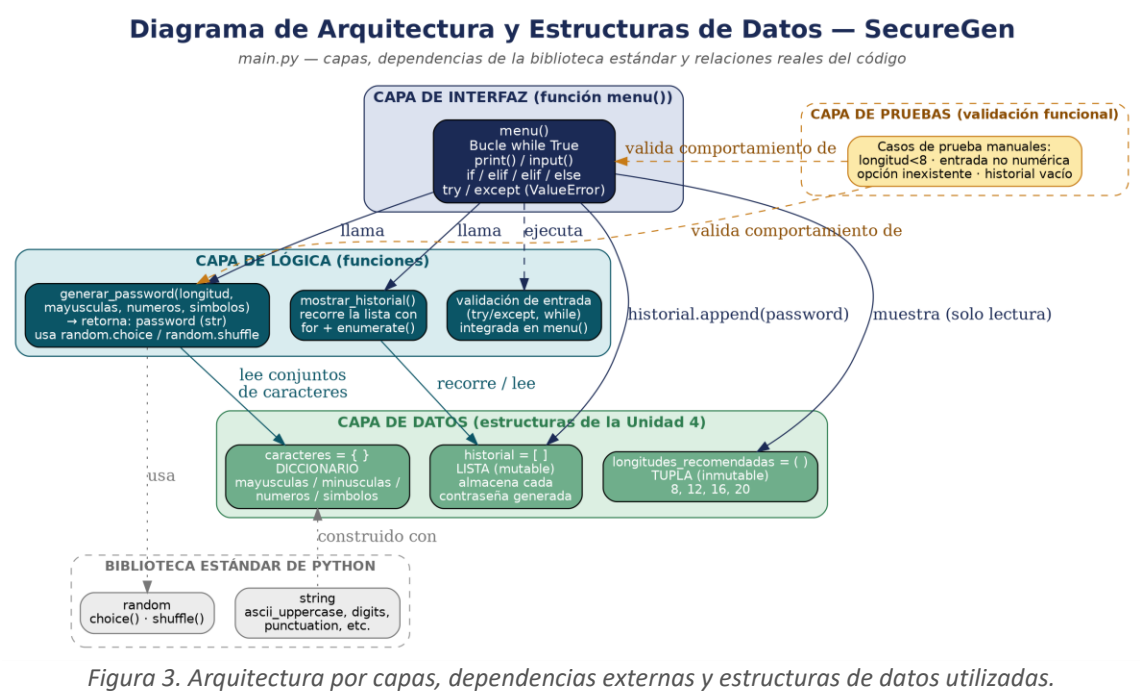


Figura 3. Arquitectura por capas, dependencias externas y estructuras de datos utilizadas.

Análisis de Software: Patrones, Diferencias y Similitudes

Con el fin de justificar por qué se utilizaron tres estructuras de datos distintas —y no una sola estructura genérica—, se organizó y sintetizó la evidencia del código para identificar patrones, diferencias y similitudes entre ellas, relacionando cada hallazgo directamente con el enfoque del proyecto (la organización flexible de configuración, lógica y datos). El resultado se resume en la Tabla 2.

Tabla 2. Patrones, diferencias y similitudes entre las estructuras de datos utilizadas

Patrón identificado	Evidencia en el código de SecureGen y relación con el enfoque
Diferencia: mutabilidad	La lista historial crece dinámicamente con cada contraseña (historial.append()), mientras que la tupla longitudes_recomendadas

Patrón identificado	Evidencia en el código de SecureGen y relación con el enfoque
	permanece fija durante toda la ejecución, ya que las tuplas son inmutables por definición. Esta diferencia refleja directamente la naturaleza de cada dato: uno cambia en tiempo de ejecución, el otro es una referencia constante.
Patrón: agrupación por clave	El diccionario caracteres organiza los cuatro conjuntos de símbolos bajo una clave descriptiva ("mayusculas", "numeros", etc.), lo que permite activar o desactivar cada grupo de forma independiente según la respuesta del usuario, sin necesidad de condicionales anidados adicionales.
Similitud: iterabilidad	Tanto la lista como el diccionario pueden recorrerse mediante bucles for (for i in historial, for valor in caracteres), lo que evidencia una característica común a varias estructuras de datos de Python, aunque cada una se itere con un propósito distinto (mostrar historial vs. construir el pool de caracteres).
Alternativa descartada	Concatenar todos los caracteres en una sola cadena de texto, sin diferenciarlos por categoría, habría impedido permitir o restringir un tipo de carácter de forma independiente y habría dificultado garantizar al menos un carácter de cada tipo seleccionado; por ello se descartó frente al uso del diccionario.

Esta síntesis permite concluir que la combinación de tupla, diccionario y lista —y no el uso de una sola estructura genérica como una cadena de texto— resultó la opción más flexible, legible y fiel a la naturaleza de cada dato dentro del problema planteado (Luján Castillo, 2019). El patrón general que emerge es que la elección de estructura de datos no fue arbitraria, sino que respondió en cada caso a si el dato necesitaba (a) cambiar durante la ejecución, (b) agruparse por una clave significativa, o (c) permanecer fijo como referencia.

Explicación del Sistema Desarrollado

SecureGen es una aplicación de consola compuesta por un único archivo (main.py) y tres funciones principales. A continuación, se detallan sus componentes.

Estructuras de Datos Implementadas

Variable	Tipo	Rol en el sistema
caracteres	Diccionario	Agrupar los conjuntos de caracteres disponibles (mayúsculas, minúsculas, números, símbolos) bajo claves descriptivas.

Variable	Tipo	Rol en el sistema
historial	Lista (mutable)	Almacena, en orden, cada contraseña generada durante la ejecución; permite agregar (append) y recorrer (for/enumerate).
longitudes_recomendadas	Tupla (inmutable)	Conserva valores fijos de referencia (8, 12, 16, 20) que no deben modificarse durante la ejecución.

Funciones Implementadas

- `generar_password(longitud, mayusculas, numeros, simbolos)`: combina los conjuntos del diccionario caracteres según las preferencias del usuario, garantiza al menos un carácter de cada tipo seleccionado y retorna una cadena (str) con la contraseña generada. Para ello recurre a las funciones `choice()` y `shuffle()` del módulo `random`, y a las constantes del módulo `string` de la biblioteca estándar de Python (Python Software Foundation, 2024).
- `mostrar_historial()`: recorre la lista `historial` mediante un bucle `for` con `enumerate()` y muestra cada contraseña generada en la sesión; si la lista está vacía, informa explícitamente al usuario en lugar de mostrar una salida vacía.
- `menu()`: función principal que controla el flujo del programa mediante un bucle `while True`, estructuras condicionales (`if/elif/else`) y manejo de excepciones (`try/except`) para validar la entrada numérica del usuario y la longitud mínima solicitada.

Estructuras Lógicas Aplicadas

- Condicionales: validan la opción del menú y las respuestas de tipo sí/no del usuario.
- Repetitivas (`while`): controlan el bucle principal del menú y el reintento ante una longitud mínima no válida.
- Repetitivas (`for`): recorren la lista `historial` para mostrarla en pantalla.
- Manejo de errores (`try/except`): evita que el programa se detenga si el usuario ingresa un valor no numérico.

Resultados

Durante la fase de pruebas se ejecutó el sistema de forma reiterada, ingresando distintas combinaciones de longitud y preferencias de caracteres, así como valores no válidos (letras en lugar de números, longitudes menores a 8, opciones de menú inexistentes) para verificar el comportamiento de las validaciones. La Tabla 3 resume los casos de prueba ejecutados y su resultado.

Tabla 3. Casos de prueba ejecutados sobre generar_password() y menu()

#	Entrada / acción	Resultado esperado	Resultado
1	Longitud = 14, mayúsc./núm./símb. = sí	Contraseña de 14 caracteres con los 4 tipos	Cumplido
2	Longitud = 5 (menor al mínimo)	Mensaje de error y nueva solicitud	Cumplido
3	Longitud = "abc" (texto, no numérico)	ValueError controlado (try/except)	Cumplido
4	Opción de menú = "9" (no existe)	Mensaje "Opción no válida"	Cumplido
5	Generar 3 contraseñas y elegir "Ver historial"	Se listan las 3, en orden, con enumerate()	Cumplido
6	"Ver historial" antes de generar ninguna contraseña	Mensaje "Historial vacío", sin error	Cumplido

En todos los casos el programa respondió de acuerdo con lo esperado: solicitó nuevamente el dato cuando este no era válido y generó una contraseña distinta en cada ejecución gracias al uso de `random.choice()` y `random.shuffle()`.

Figura 4. Evidencia de ejecución: generación de una contraseña y consulta del historial.

(Ver evidencia de ejecución en consola adjunta al proyecto.)

Como resultado directo del desarrollo, se obtuvo un sistema funcional capaz de: (1) generar contraseñas de longitud variable que cumplen con criterios mínimos de seguridad (combinación de tipos de caracteres y garantía de al menos uno de cada tipo seleccionado), (2) registrar y mostrar el historial de contraseñas generadas en la sesión, incluyendo el caso de historial vacío, y (3) manejar entradas inválidas —tanto numéricas como de opción de menú— sin interrumpir la ejecución.

Estos resultados responden de manera específica a los objetivos planteados y permiten una extrapolación lógica acotada a su alcance: la misma estrategia de organización aplicada aquí —separar configuración (diccionario), historial mutable (lista) y referencias fijas (tupla) en estructuras de datos distintas, y aislar la validación de entrada en bloques `try/except` reutilizables— es directamente aplicable a otros sistemas de consola que deban administrar múltiples categorías de configuración con validación de entrada robusta, como un gestor de tareas, un sistema de encuestas o un validador de formularios de texto. Esta extrapolación se limita deliberadamente a sistemas de complejidad y escala comparable a SecureGen, y no pretende generalizarse a problemas de ciberseguridad de mayor escala (por ejemplo, gestión de

credenciales empresariales o autenticación multifactor), que exceden el alcance verificado por las pruebas realizadas.

Implicaciones y Limitaciones

Implicaciones

- Contar con una herramienta simple para generar contraseñas seguras puede contribuir, a pequeña escala, a reducir prácticas de riesgo evidenciadas en las pruebas, como el uso de longitudes insuficientes o la omisión de ciertos tipos de caracteres, ya que el sistema obliga a decidir explícitamente sobre ambos aspectos antes de generar la contraseña.
- La separación verificada entre capa de interfaz, lógica y datos (Figura 3) demuestra en código — no solo en teoría— que conceptos básicos de programación pueden traducirse en una solución con utilidad real y mantenible.
- El manejo de excepciones probado en la Tabla 3 (casos 2 y 3) evidencia que un sistema introductorio puede ser robusto frente a errores de usuario sin requerir frameworks externos.

Limitaciones

- El sistema actual no almacena el historial de forma permanente: la lista historial se reinicia en memoria cada vez que el programa se cierra, por lo que no existe persistencia entre sesiones (limitación evidenciada directamente por el tipo de dato lista, que no se serializa a disco en la implementación actual).
- El módulo `random` utilizado en `generar_password()` no es criptográficamente seguro; para un entorno de producción real se recomendaría el módulo `secrets` de Python, una limitación que no se manifestó en las pruebas funcionales pero que es relevante para el uso real de las contraseñas generadas.
- El sistema no verifica si la contraseña generada ha sido filtrada previamente en bases de datos públicas de contraseñas comprometidas, ni contrasta el resultado contra listas de credenciales vulneradas, una práctica que el NIST recomienda explícitamente para los sistemas de autenticación actuales (National Institute of Standards and Technology [NIST], 2025).
- Su interfaz es exclusivamente de consola, lo que limita su uso por parte de personas no familiarizadas con este tipo de entornos, una limitación de usabilidad que no fue parte del alcance definido para este proyecto pero que condiciona su adopción fuera de un contexto académico.

- La fortaleza final de la contraseña depende de las opciones que el propio usuario elija (por ejemplo, si decide no incluir símbolos ni números); el sistema valida la longitud mínima pero no impone un mínimo de tipos de caracteres.

Estas limitaciones, evidenciadas directamente a partir del comportamiento observado del código y no de supuestos externos, no invalidan el cumplimiento del objetivo del proyecto, pero sí delimitan claramente su alcance y abren posibilidades concretas de mejora a futuro, como el almacenamiento cifrado del historial, el uso del módulo secrets, una interfaz gráfica o la verificación de la contraseña contra listas de credenciales filtradas conforme a los lineamientos vigentes de identidad digital (NIST, 2025).

Reflexión sobre el Impacto de la Tecnología en la Sociedad

El desarrollo de este proyecto permitió reflexionar sobre una idea central: la tecnología no solo crea nuevas oportunidades, sino también nuevas responsabilidades. A medida que más actividades cotidianas —estudiar, trabajar, socializar, realizar trámites bancarios— dependen de plataformas digitales, la seguridad de la información personal deja de ser un tema exclusivo de especialistas en informática y se convierte en una necesidad de toda la sociedad.

Asimismo, este proyecto evidencia que no se requieren herramientas complejas para generar un impacto positivo: una aplicación sencilla, bien estructurada y fundamentada en buenas prácticas de programación, puede ayudar a las personas a protegerse de uno de los vectores de ataque más comunes en la actualidad. En ese sentido, la formación en programación —y específicamente en estructuras de datos y funciones— no es solo un contenido académico, sino una competencia con proyección directa hacia la solución de problemas reales de la sociedad.

De cara al futuro, se proyecta que herramientas como SecureGen evolucionen hacia gestores de contraseñas más completos, integrados con verificación de filtraciones de datos y cifrado robusto, en línea con las recomendaciones vigentes de organismos como el NIST (2025), lo cual demuestra que un proyecto académico puede ser el punto de partida de soluciones tecnológicas con mayor alcance.

Conclusiones

- Se identificó un problema acotado, vigente y verificable —el uso de contraseñas inseguras— y se desarrolló una solución informática funcional para abordarlo, evitando deliberadamente un alcance demasiado amplio.

- Se aplicaron de forma integrada los contenidos de las cuatro unidades de la asignatura, con especial profundidad en tuplas, listas, diccionarios y funciones.
- El análisis comparativo de las estructuras de datos confirmó patrones de mutabilidad, agrupación e iterabilidad que justifican su elección frente a alternativas más simples como una única cadena de texto.
- Las pruebas realizadas (Tabla 3) confirman que el sistema cumple con los criterios de funcionamiento definidos en la planificación del proyecto, y permiten extrapolar su lógica de diseño a problemas de escala y complejidad comparables, sin generalizar más allá de lo verificado.
- Se reconocieron limitaciones concretas y evidenciadas del sistema (ausencia de persistencia, uso de random no criptográfico, falta de verificación contra filtraciones), lo que demuestra una evaluación crítica y realista del trabajo desarrollado.
- El proyecto permitió reflexionar sobre la relación entre la formación en programación y su aplicación en problemáticas reales de la sociedad actual.

Referencias

Luján Castillo, M. (2019). Estructuras de datos en Python. Material de la asignatura, Unidad 4: Estructura de Datos y Funciones, UIDE.

National Institute of Standards and Technology. (2025). Digital identity guidelines: Authentication and authenticator management (NIST SP 800-63B, Rev. 4). U.S. Department of Commerce.

<https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-63B-4.pdf>

Python Software Foundation. (2024). The Python standard library. <https://docs.python.org/3/library/>

UIDE. (2026). Guía del proyecto integrador: Programación [Documento institucional de la asignatura].